

CS 425 / ECE 428

Distributed Systems

Fall 2026

Aishwarya Ganesan

w/ Ram Kesavan

(Thanks to Indranil Gupta)

Lecture 3: Mapreduce and Hadoop

What is MapReduce?

- Terms are borrowed from Functional Language (e.g., Lisp)

Sum of squares:

- (square 2)
 - Output: 4
- (map square '(1 2 3 4))
 - Output: (1 4 9 16)
 - [processes each record independently]
- (reduce + '(1 4 9 16))
 - (+ 16 (+ 9 (+ 4 1)))
 - Output: 30
 - [processes set of all records in batches]

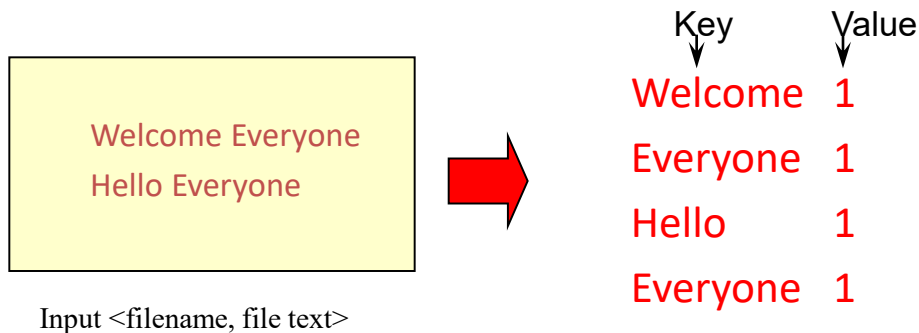
Real Application

Let's consider a sample application: **Wordcount**

- You are given a huge dataset
 - (e.g., Wikipedia dump or all of Shakespeare's works)
- List the count for each of the words in each of the documents in the dataset

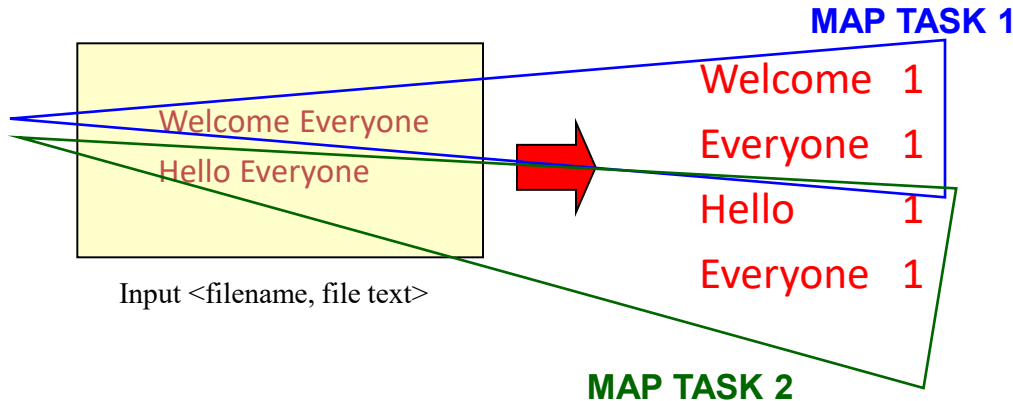
Map

- Process individual records to generate intermediate key/value pairs.

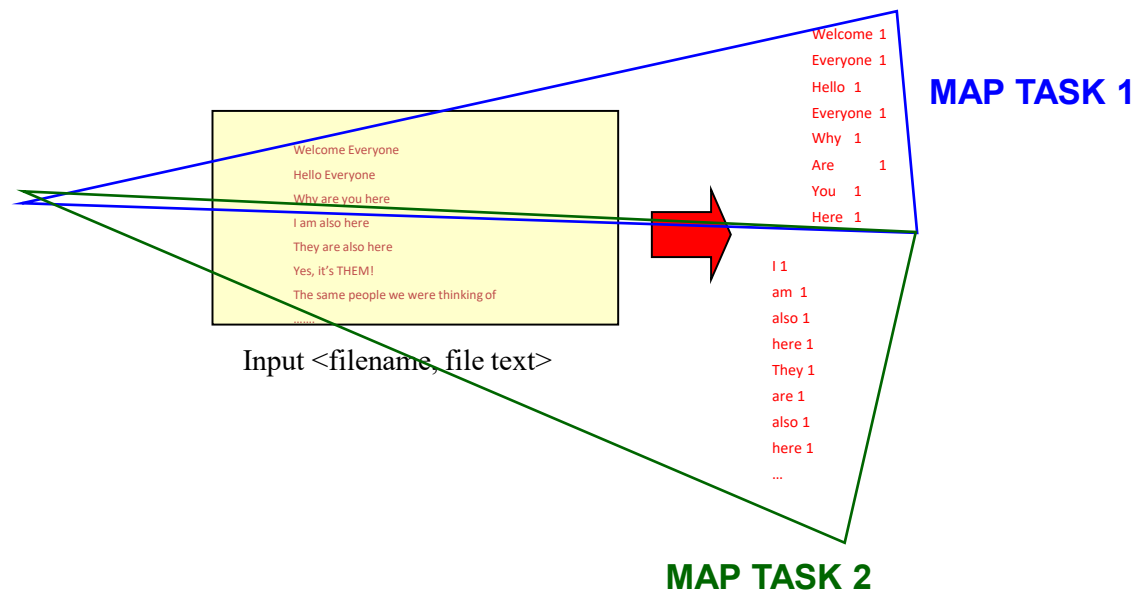


Map

- **Parallelly** Process individual records to generate intermediate key/value pairs.



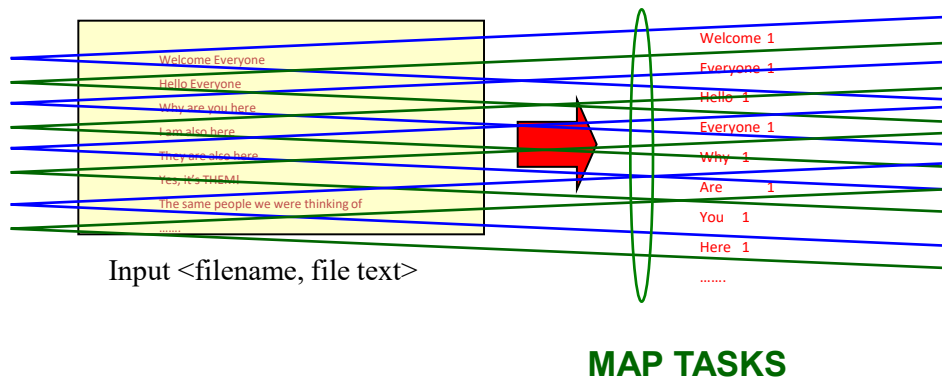
Map



- Map task invokes map function once per input key-value pair

Map

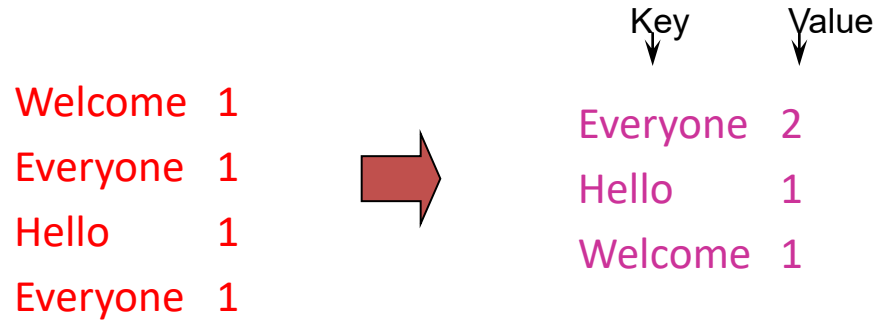
- **Parallelly** Process **a large number** of individual records to generate intermediate key/value pairs.



Same set of intermediate KV pairs
no matter how you parallelize it.

Reduce

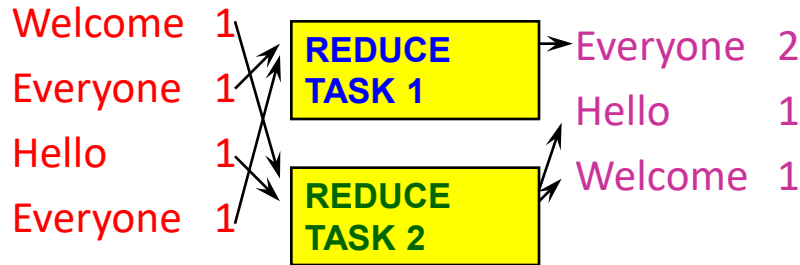
- Reduce processes and merges all intermediate values associated per key



How to **parallelize** reduce?

Reduce

- Each key assigned to one Reduce
- Parallelly Processes and merges all intermediate values by partitioning keys



- Reduce task invokes reduce function once per intermediate key, list of values
- Popular: *Hash partitioning*, i.e., key is assigned to
 - $\text{reduce \#} = \text{hash}(\text{key}) \% \text{number of reduce tasks}$

Map and Reduce functions for WordCount

map (key, value):

```
// key is empty, value is the line
```

```
    for each word in value:
```

```
        output(word,1)
```

reduce (key, values):

```
// key is word, value is list of integers
```

```
    sum = 0
```

```
    for each value in values:
```

```
        sum+=value
```

```
    output(key, sum)
```

Hadoop Code - Map

```
public static class TokenizerMapper
    extends Mapper<Object, Text, Text, IntWritable>{

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context
        ) throws IOException, InterruptedException {
        // key is empty, value is the line
        // invoked for every line
        StringTokenizer itr = new StringTokenizer(value.toString());
        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}
```

Hadoop Code - Reduce

```
public static class IntSumReducer
    extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();
    public void reduce(
        Text key,
        Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException {
        // key is word, values is a list of 1's
        // called exactly once for each key (e.g., "hello")

        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```

Hadoop Code - Driver

```
// Tells Hadoop how to run your Map-Reduce job
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    // The job.
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setReducerClass(IntSumReducer.class);
    // The keys are words
    job.setOutputKeyClass(Text.class);
    // The values are counts (ints)
    job.setOutputValueClass(IntWritable.class);
    for (int i = 0; i < otherArgs.length - 1; ++i) {
        FileInputFormat.addInputPath(job, new Path(otherArgs[i]));
    }
    FileOutputFormat.setOutputPath(job,
        new Path(otherArgs[otherArgs.length - 1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

Some Applications of MapReduce

Distributed Grep:

- Input: large set of files
- Output: lines that match pattern
- Map – *Emits a line if it matches the supplied pattern*
- Reduce – *Copies the intermediate data to output*

Some Applications of MapReduce

(2)

Reverse Web-Link Graph

- Input: Web graph: tuples (a, b) where (page $a \rightarrow$ page b)
- Output: For each page, list of pages that link *to* it
- Map – *process web log and for each input $\langle source, target \rangle$, it outputs $\langle target, source \rangle$*
- Reduce - *emits $\langle target, list(source) \rangle$*

Some Applications of MapReduce

(3)

Word with maximum count

- Input: You are given a huge dataset
- Output: Word that appears the most in the dataset

Start with MR similar to WordCount:

- Map – *Process each word in line and outputs <word, 1>*
- Multiple Reducers - *Emits <word, word_count>*

Can you maintain maximum word count across all reduce function invocations?
Why or why not?

Some Applications of MapReduce

(3)

Word with maximum count

- Input: You are given a huge dataset
- Output: Word that appears the most in the dataset

Start with MR similar to WordCount:

- Map – *Process each word in line and outputs $\langle \text{word}, 1 \rangle$*
- Multiple Reducers - *Emits $\langle \text{word}, \text{word_count} \rangle$*
- Chain another MapReduce job after above one
- Map – *Processes $\langle \text{word}, \text{word_count} \rangle$ and outputs $\langle 1, (\langle \text{word}, \text{word_count} \rangle) \rangle$*
- 1 Reducer – Gets the list of all *$\langle \text{word}, \text{word_count} \rangle$ pairs*
 - Find the word corresponding to the max count in the list.

Why not have just one MR instead of chaining and have the first map emit $\langle 1, (\langle \text{word}, 1 \rangle) \rangle$?

Some Applications of MapReduce

(4)

Count of URL access frequency

- Input: Log of accessed URLs, e.g., from proxy server
- Output: For each URL, % of total accesses for that URL

Start with MR similar to WordCount:

- Map – *Process web log and outputs <URL, 1>*
- Multiple Reducers - *Emits <URL, URL_count>*

You have counts, how to get percentages?

Some Applications of MapReduce

(4)

Count of URL access frequency

- Input: Log of accessed URLs, e.g., from proxy server
- Output: For each URL, % of total accesses for that URL

Start with MR similar to WordCount:

- Map – *Process each word in line and outputs <word, 1>*
- Multiple Reducers - *Emits <word, word_count>*

(You have counts, how to get percentages?)

- Chain another MapReduce job after above one
- Map – *Processes <URL, URL_count> and outputs <1, (<URL, URL_count>)>*
- 1 Reducer – Does two passes.
 - In first pass, sums up all *URL_count's* to calculate overall_count.
 - In second pass calculates %'s

Emits multiple <URL, URL_count/overall_count>

Some Applications of MapReduce

(5)

Map task's output is sorted (e.g., quicksort)

Reduce task's input is sorted (e.g., mergesort)

Sort

- Input: Series of (key, value) pairs
- Output: Sorted <value>s
- Map – $\langle \text{key}, \text{value} \rangle \rightarrow \langle \text{value}, _ \rangle$ (*identity*)
- Reducer – $\langle \text{key}, \text{value} \rangle \rightarrow \langle \text{key}, \text{value} \rangle$ (*identity*)
- Partitioning function – partition keys across reducers based on **ranges** (can't use hashing!)
 - Take data distribution into account to balance reducer tasks

Announcements

- **4 cr only: MP Groups Form closed now**
- **DO NOT**
 - Change MP groups unless your partner has dropped
 - Leave your MP partner hanging: Both MP partners should contribute equally (we will ask!)
- **All: Please fill out Student Survey (see course webpage).**
- MP1 due Sep 13th
 - VMs being distributed soon (watch Piazza)
 - Demos will be Monday Sep 14th (schedule and details will be posted soon on Piazza)
- HW1 out today due Sep 20th: Solve problems right after lecture covers topic!
- **Check Piazza often! It's where all the announcements are at!**

CS 425 / ECE 428
Distributed Systems
Fall 2026

Aishwarya Ganesan
w/ Ram Kesavan

Mapreduce and Hadoop (contd)

Inside MapReduce

In-class activities

- Quizzes vs. Polls
- Polls: anonymous, not mandatory, not graded
 - We might use external tools
 - Will use poll everywhere. <https://pe.app/aishwaryaganesa>
- Quizzes: 4-5 over the semester
 - Graded
 - Canvas



Suppose the mappers produce:
(cat, 1) (dog, 1) (cat, 1) (bird, 1) (dog, 1)
Which of the following is guaranteed?

 Responses hidden

All five records go to the same reduce task

All records for cat go to the same reduce task

Each map task sends all of its output to exactly one reduce task

Cat records may go to different reduce tasks, and Hadoop combines them later



 Anonymous 192

 Edit

☒ Choices

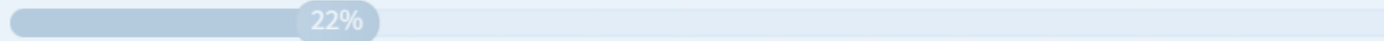
☐ Results

☐ Lock

☐ Correctness



How many times is reduce() function called?
Suppose the complete mapper output is:
(cat,1) (cat,1) (cat,1) (dog,1) (dog,1)



5



2



Depends on number of reduce task



Programming MapReduce

Externally: For **user**

1. Write a Map program (short), write a Reduce program (short)
2. Specify number of Maps and Reduces (parallelism level) and partitioning function
3. Submit job; wait for result
4. Need to know very little about parallel/distributed programming!

Programming MapReduce

Externally: For **user**

1. Write a Map program (short), write a Reduce program (short)
2. Specify number of Maps and Reduces (parallelism level)
3. Submit job; wait for result
4. Need to know very little about parallel/distributed programming!

Internally: For the Paradigm and Scheduler

1. Parallelize Map
2. Transfer data from Map to Reduce (**shuffle data**)
3. Parallelize Reduce
4. Implement Storage for Map input, Map output, Reduce input, and Reduce output

(Ensure that no Reduce starts before all Maps are finished. That is, ensure the **barrier** between the Map phase and Reduce phase) Why?

Inside MapReduce

1. Parallelize Map: **easy!** each map task is independent of the other!
2. Transfer data from Map to Reduce:
3. Parallelize Reduce: **easy!** each reduce task is independent of the other!
4. Implement Storage for Map input, Map output, Reduce input, and Reduce output
 - Map input: from **distributed file system**
 - Map output: to local disk (at Map node); uses **local file system**
 - Reduce input: from (multiple) remote disks; uses local file systems (why write to disk?)
 - Reduce output: to distributed file system
 - One output file per reduce task

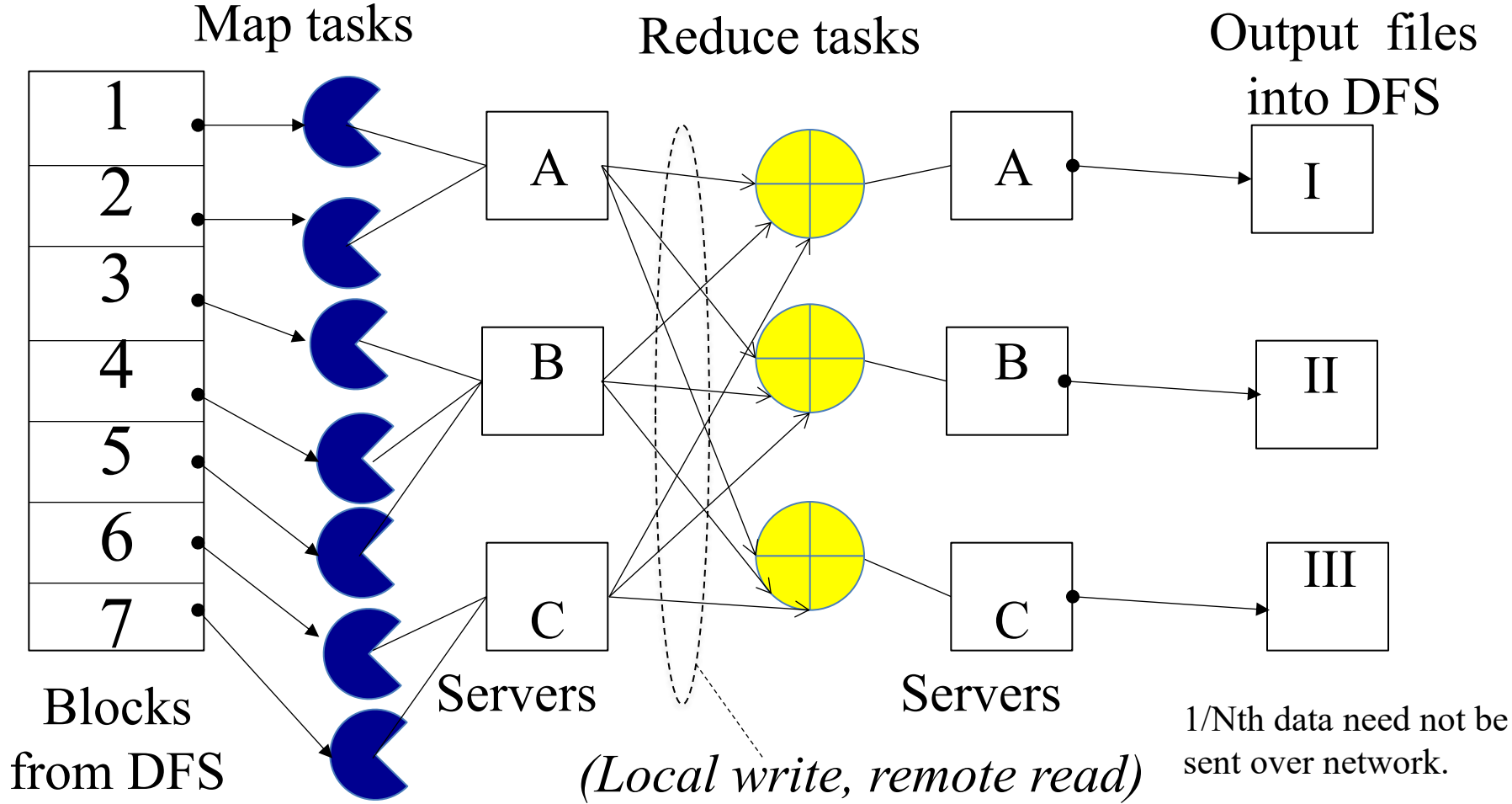
local file system = Linux FS, etc.

distributed file system = GFS (Google File System), HDFS (Hadoop Distributed File System)

Inside MapReduce

1. Parallelize Map: **easy!** each map task is independent of the other!
2. Transfer data from Map to Reduce:
 - Called Shuffle data
 - All Map output records with same key assigned to same Reduce task
 - Map task's output is sorted (e.g., quicksort)
 - Use **partitioning function, e.g., $\text{hash}(\text{key})\% \text{number of reducers}$ or range partitioning**
 - Each map task creates R partitions
 - Reduce task's input is sorted (merge the intermediate outputs from M map tasks)
 - Ensure **barrier** between the Map phase and Reduce phase
3. Parallelize Reduce: **easy!** each reduce task is independent of the other!
4. Implement Storage for Map input, Map output, Reduce input, and Reduce output

Same set of machines store DFS file blocks, and can run map/reduce tasks

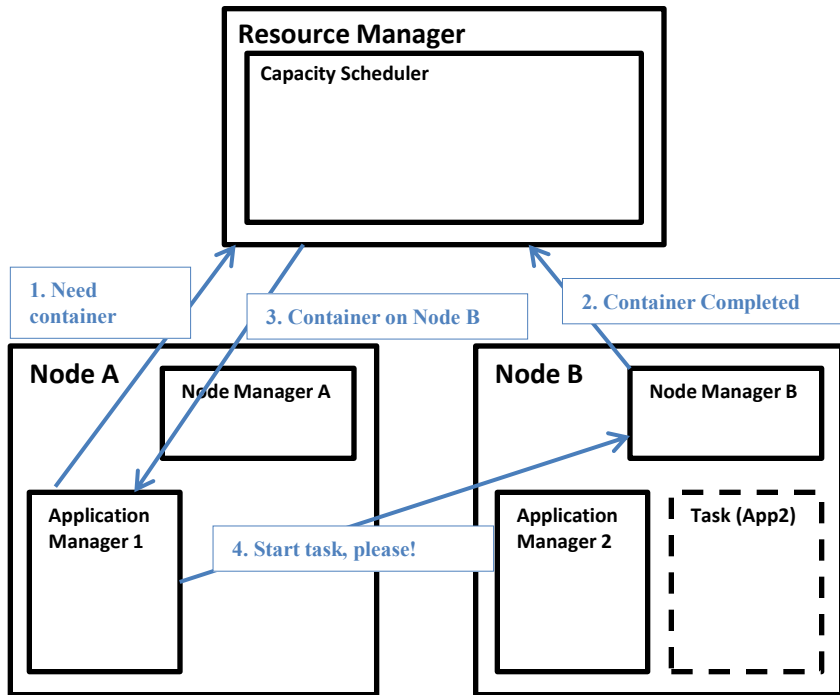


Resource Manager (assigns maps and reduces to servers)

The YARN Scheduler

- Used underneath Hadoop 2.x +
- YARN = Yet Another Resource Negotiator
- Treats each server as a collection of logical *containers*
 - Container = fixed CPU + fixed memory (think of Linux cgroups, but even more lightweight)
- Has 3 main components
 - Global *Resource Manager (RM)*
 - Scheduling
 - Per-server *Node Manager (NM)*
 - Daemon and server-specific functions
 - Keep track of containers/tasks on that machine
 - Per-application (job) *Application Manager (AM)*
 - Container negotiation with RM and NMs
 - Detecting task failures of that job

YARN: How a job gets a container



- *2 servers (A, B)*
- *2 jobs (1, 2)*

Fault Tolerance

- Server Failure
 - NM heartbeats to RM
 - If server fails: RM times out waiting for next heartbeat
 - RM lets all affected AMs know
 - AMs take appropriate action (restart tasks on different machines)
- Task Failure
 - NM keeps track of each task running at its server
 - If task fails while in-progress, mark the task as idle and restart it
- Why safe to restart failed map task?
- Reduce task? What if you wrongly detect and start another?

Write to temporary file and rename at end

Fault Tolerance (cont.)

- AM Failure
 - AM heartbeats to RM
 - On failure, RM restarts AM
 - Syncs it up with its running tasks
- RM Failure
 - Use old checkpoints and bring up secondary RM
- Heartbeats also used to piggyback container requests
 - Avoids extra messages

Slow Servers

Slow tasks are called **Stragglers**

- The slowest task slows the entire job down (why?)
- Why will a task be slow?
- Keep track of “progress” of each task (% done)
- Perform proactive backup (replicated) execution of straggler tasks
 - A task done when its first replica complete (other replicas can be killed)
 - Approach called **Speculative Execution**.
 - Why is it safe?
 - How will a restarted reduce task get inputs from the map task?

Barrier at the end
of Map phase!

Locality

- The same cluster stores DFS blocks, run map tasks, and run reduce tasks
- For server-fault-tolerance, GFS/HDFS stores 3 replicas of each file chunk (e.g., 64 MB in size)
 - Since cloud has hierarchical topology (e.g., racks)
 - For rack-fault-tolerance, on different racks, e.g., 2 on a rack, 1 on a different rack
- Locality
 - Mapreduce attempts to schedule a map task on
 - a machine that contains a replica of corresponding input data, or failing that,
 - on the same rack as a machine containing the input, or failing that,
 - Anywhere

Note: The 2-1 split of replicas is intended to reduce bandwidth when writing file.

That was Hadoop 2.x...

- Hadoop 3.x (new!) over Hadoop 2.x
 - Dockers instead of container
 - Erasure coding instead of 3-way replication
 - Multiple Namenodes instead of one (name resolution)
 - GPU support (for machine learning)
 - Intra-node disk balancing (for repurposed disks)
 - Intra-queue preemption in addition to inter-queue
 - (<https://hadoop.apache.org/docs/r3.0.0/>)

Mapreduce: Summary

- Mapreduce uses parallelization + aggregation to schedule applications across clusters
- Need to deal with failure
- Plenty of research in scheduling and fault-tolerance for Mapreduce and Hadoop

Further MapReduce Exercises

Exercise 1

1. (MapReduce) You are given a symmetric social network (like Facebook) where a is a friend of b implies that b is also a friend of a . The input is a dataset D (sharded) containing such pairs (a, b) – note that either a or b may be a lexicographically lower name. Pairs appear exactly once and are not repeated. **Find the last names of those users whose first name is “Ye” and who have at least 300 friends. You can assume full names are unique.** You can chain Mapreduces if you want (but only if you must, and even then, only the least number). You don’t need to write code – pseudocode is fine as long as it is understandable. Your pseudocode may assume the presence of appropriate primitives (e.g., “firstname(user_id)”, etc.). The Map function takes as input a tuple (key= a , value= b).

Exercise 1: Solution

Goal: Last names of those users whose first name

- M1 (a,b): *is “Ye” and who have at least 300 friends.*
 - if (firstname(a)==Ye) then output (a,b)
 - if (firstname(b)== Ye) then output (b,a)
 - // note that second if is NOT an else if, so a single M1 function may be output up to 2 KV pairs!
- R1 (x, V):
 - if $|V| \geq 300$ then output (lastname(x), -)

Exercise 2

2. For an asymmetrical social network, you are given a dataset D where lines consist of (a,b) which means user a follows user b.

Write a MapReduce program (Map and Reduce separately) that outputs the list of all users U who satisfy the following three conditions simultaneously: i) user U has at least 2 million followers, and ii) U follows fewer than 20 other users, and iii) all the users that U follows, also follow U back.

Exercise 2: Solution

Goal: Find users U

- $M1(a,b)$:
 - Output (key=a, value=(OUT,b))
 - Output (key=b, value=(IN,a))
 - // Note that a single M1 function outputs exactly TWO KV pairs
- $R1(\text{key}=x, V)$:
 - Collect Sout = set of all (OUT,*) value items from V
 - Collect Sin = set of all (IN,*) value items from V
 - if ($|Sout| < 20$ AND $|Sin| \geq 2M$ AND all items in Sout are also present in Sin) // third term via nested for loops
 - then output (x,_)

Exercise 3

3. For an asymmetrical social network, you are given a dataset D where lines consist of (a,b) which means user a follows user b . Write a MapReduce program (Map and Reduce separately) that outputs the **list of all user *pairs* (x,y) who satisfy the following three conditions simultaneously**: i) x has fewer than 100 M followers, ii) y has fewer than 100M followers, iii) x and y follow each other, and iv) *the sum* of x 's followers and y 's followers (double-counting common followers that follow both x and y is ok) is 100 M or more. Your output should not contain duplicates (i.e., no (x,y) and (y,x)).

Exercise 3: Solution

Goal: find pairs (x,y):

- $M1(a,b)$: output (b,a)
- $R1(x,V)$:
 - if $|V| < 100M$, then for all a in V , output $(\text{lexicographic_sorted_pair}(x,a), |V|)$
- $M2(a,b)$: Identity
- $R2(\text{key}=(a,b), \text{value}=\{|V1|, |V2|, \dots\})$
 - if $|\text{value}|==1$ output nothing // they don't follow each other
 - else if $|\text{value}|==2$ then add value entries // follow each other
 - if sum of value entries $\geq 100M$ then output (a,b)

i) x has $< 100 M$ followers,

ii) y has $< 100M$ followers,

iii) x and y follow each other,

iv) sum of x 's & y 's followers $\geq 100 M$
(double count ok).

Exercise 4

4. You are given a symmetric social network (like Facebook) where a is a friend of b implies that b is also a friend of a . The input is two datasets $D1$ and $D2$. $D1$ contains pairs (a, b) .

$D2$: contains pairs $(\text{home_address}, \text{full name})$. If multiple people live in an address you will have multiple entries.

Find the names and addresses of those users whose first name is “Ye” and who have at least 300 friends. You can assume full names are unique.

Exercise 4: Solution (cont.)

*Goal: Full names and addresses of users
with first name “Ye” and at least 300 friends.*

- `//reads dataset D1`
- `M1 (a,b):`
 - `if (firstname(a)==Ye) then output (a,b)`
 - `if (firstname(b)== Ye) then output (b,a)`
- `R1 (x, V):`
 - `if |V| >= 300 then output ( x, <“USER_TAG”,_>)`

Exercise 4: Solution (cont.)

//reads dataset D2

- M2 (address, user):
 - if (firstname(user)==Ye) then
 - output (user, <“ADDRESS_TAG”, address>)
- R2: Identity

Exercise 4: Solution (cont.)

- //Input to MR3 is outputs of MR1 and MR2
- M3:Identity
- R3: (user, values):
 - if len(values) == 2 then
 - address = second entry of tuple in values whose first entry is “ADDRESS_TAG”
 - output (user, address)

Remember ...

- Same map task cannot read two input datasets
 - Use two MR jobs to create inputs to 3rd MR job
 - MR job can take multiple input files (See exercise 4)
- Reduce cannot read arbitrary input data or use a global variable
- You cannot skip map or reduce phases
- You cannot use intermediate data from a map in a subsequent MR phase in the chain

Announcements

- **MP1: Due Sunday 9/13, demos Monday 9/14**
 - VMs distributed: see Piazza
 - GitHub course repo: see Piazza
 - Demo signup sheet: tomorrow on Piazza
 - (see signup deadline – this Friday!)
 - Demo details: tomorrow on Piazza
- **HW1: due 9/20**
- Check Piazza often! It's where all the announcements are at!